

13강

spring MVC MyBatis





➤ MyBatis 란

- SQL Mapping Framework – Easy & Simple
- 매개변수 설정과 쿼리 결과를 읽어오는 코드를 제거한다.
- 자바 코드로부터 SQL문을 분리해서 관리한다.
- 작성할 코드가 줄어서 생산성 향상 & 유지 보수 편리하다
- MyBatis는 SQL을 XML(또는 어노테이션)에 적어두고 그 결과를 자바 객체로 자동 변환해주는 도구
- SQL을 쉽게 다룰 수 있도록 도와주는 프레임워크이다.

➤ 매퍼란?

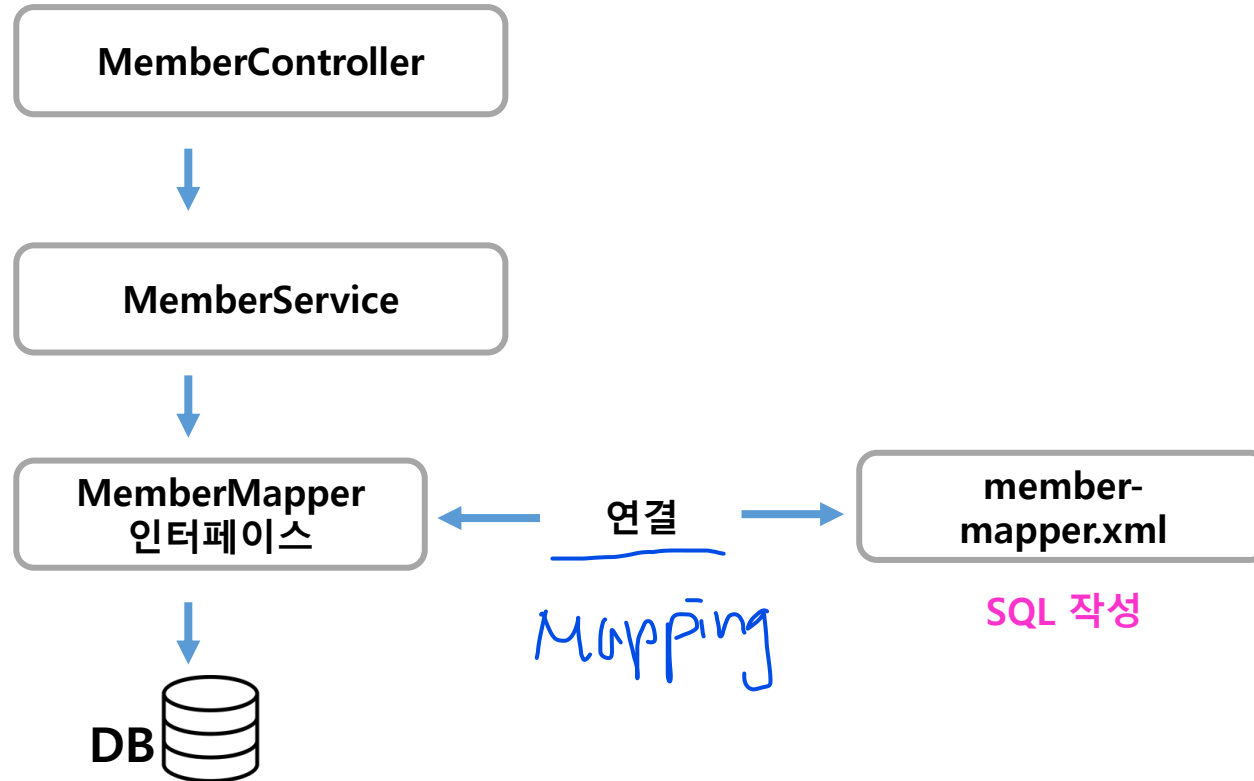
- SQL과 Java 객체를 연결해 주는 역할

➤ 매퍼 인터페이스 vs. XML 매퍼

- 매퍼 인터페이스(UserMapper): 메서드만 정의(SQL 실행 X)
- XML 매퍼: 실제 SQL이 작성된 파일(SQL 실행 O)



➤ MyBatis 전체 구조 한 눈에 보기





➤ MyBatis – Mapper & xml

MemberMapper 인터페이스

@Mapper

```
public interface MemberMapper {  
    public int insertMember(MemberDTO mdto);  
}
```

① 호출

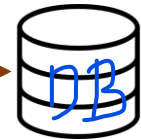
이 SQL로 들어오는 값은
👉 MemberDTO 한 덩어리다

② 연결(mapping)

member-mapper.xml

```
<mapper namespace="com.green.member.mapper.MemberMapper">  
    <insert id="insertMember"  
        parameterType="com.green.member.MemberDTO">  
        INSERT INTO user_member(id,pw,mail,phone)  
        VALUES(#{id},#{pw},#{mail},#{phone})  
    </insert>  
</mapper>
```

③ 실행





➤ MyBatis 문법

문법	의미
<u>@Mapper</u>	이 인터페이스가 MyBatis Mapper라는 표시
<u>메서드 이름</u>	XML의 SQL id와 <u>반드시 동일</u>
<u>반환타입</u>	SQL 결과를 어떻게 받을지
<u>매개변수</u>	SQL로 전달할 값

MemberMapper 인터페이스

```
@Mapper
public interface MemberMapper {
    public boolean isMember(String id);
}
```

member-mapper.xml

```
<mapper
namespace="com.green.member.mapper.MemberMapper">

    <select id="isMember"
        parameterType="String"
        resultType="boolean">

        SELECT COUNT(*) FROM user_member WHERE id = #{id}

    </select>

</mapper>
```



➤ MyBatis 문법

- namespace는 이 SQL들이 어느 Mapper랑 연결될지 알려주는 주소

문법	설명
<mapper>	이 파일은 MyBatis SQL 모음
namespace	<u>Mapper 인터페이스 경로와 반드시 같아야 함</u>

MemberMapper 인터페이스

```
package com.green.member.mapper;  
  
import java.util.List;  
  
@Mapper  
public interface MemberMapper {
```

member-mapper.xml

```
<mapper  
  namespace="com.green.member.mapper.MemberMapper">  
  
  <select id="isMember"  
    parameterType="string"  
    resultType="boolean">  
  
    SELECT COUNT(*) FROM user_member WHERE id = #{id}  
  
  </select>  
  
</mapper>
```



➤ MyBatis 문법

문법	의미
<select>	조회 SQL
id	Mapper 메서드 이름
resultType	한 행을 어떤 DTO로 받을지

🔊 반환 타입 매칭 규칙

List<DTO> → resultType

DTO 1개 → resultType

int, String → resultType="int" 등

- DTO의 getter 이름을 자동으로 찾음
- DTO를 통째로 넘기면, MyBatis가 내부 값을 꺼내준다
- `dto.getName() → #{name}`
- `dto.getAge() → #{age}`

MemberMapper 인터페이스

```
@Mapper
public interface MemberMapper {
    public List<MemberDTO> allSelectMember();
}
```

member-mapper.xml

```
<mapper
namespace="com.green.member.mapper.MemberMapper">

<select id="allSelectMember"
resultType="com.green.member.MemberDTO">
>
    SELECT * FROM user_member
</select>

</mapper>
```

이 경로에서
찾음



➤ MyBatis - insert / update / delete 문법

MemberMapper 인터페이스

@Mapper

public interface MemberMapper {

```
<insert id="insertMember"
parameterType="com.green.member.MemberDTO">
    INSERT INTO user_member(id,pw,mail,phone) VALUES(#{id},#{pw},#{mail},#{phone})
</insert>

<delete id="deleteMember"
parameterType="String"
>
    DELETE FROM user_member WHERE id=#{id}
</delete>

<update id="updateMember"
parameterType="com.green.member.MemberDTO"
>
    UPDATE user_member SET mail=#{mail},phone=#{phone} WHERE id=#{id}
</update>
}
```

- **insert / update / delete**는 SQL 실행 결과를 객체로 매핑하지 않기 때문에 resultType을 사용하지 않는다
- DB 결과 영향 받은 행의 개수 이 값은 MyBatis가 **자동으로 int로 반환** 매핑할 객체가 없다. 그래서 resultType 자체를 쓰지 않는다.



spring

➤ MyBatis - #{ }와 \${ }의 차이

```
<insert id="insert" parameterType="BoardDto">
  INSERT INTO board
    (title, content, writer)
  VALUES
    (#{title}, #{content}, #{writer})
</insert>
```

```
String sql = "INSERT INTO board "
    + " (title, content, writer) "
    + "VALUES "
    + " (?, ?, ?)";
```

```
PreparedStatement pstmt = conn.prepareStatement(sql);
int result = pstmt.executeUpdate();
```

```
<insert id="insert" parameterType="BoardDto">
  INSERT INTO board
    (title, content, writer)
  VALUES
    ('${title}', '${content}', '${writer}')
</insert>
```

```
String sql = "INSERT INTO board "
    + " (title, content, writer) "
    + "VALUES "
    + " ('"+title+"', '"+content+"', '"+writer+"')";
```

```
Statement stmt = conn.createStatement();
int result = stmt.executeUpdate(sql);
```



➤ com.green프로젝트 복사 붙여넣기 하기

- com.green프로젝트를 복사해서 com.green_MyBatis로 이름을 변경하여 붙여넣기 한다.
- 이름을 변경한 후 반드시 settings.gradle를 아래와 같이 수정하고 저장한다.

```
MemberService.java settings.gradle ×  
1 rootProject.name = 'com.green_MyBatis'
```

- application.properties 도 아래와 같이 수정하고 저장한다.

```
application.properties ×  
1 spring.application.name=com.green_MyBatis
```

- build.gradle 도 아래와 같이 수정하고 저장한다.

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '4.1.0-M1'  
    id 'io.spring.dependency-management' version '1.1.7'  
}  
  
group = 'com.green_MyBatis'
```

※ 반드시 프로젝트 폴더 위에서 → 우 클릭 → Gradle → Refresh gradle project 를 실행한다.



- Mybatis 의존성 추가
- mybatis spring boot

The screenshot shows the Maven Repository search results for the query 'mybatis spring boot'. The search results show 45465 results. The first result is 'MyBatis Spring Boot Starter' by org.mybatis.spring.boot, version 4.0.1. The description states: 'Spring Boot starter for MyBatis that simplifies integration of MyBatis SQL mapping framework in Spring Boot applications. Last Release on Dec 28, 2025'. The screenshot also shows a snippet of the Maven build file configuration for the starter.

Maven Repository: mybatis sp x +

mvnrepository.com/search?q=mybatis+spring+boot

MyBatis Spring Boot Starter

org.mybatis.spring.boot » mybatis-spring-boot-starter

Spring Boot starter for MyBatis that simplifies integration of MyBatis SQL mapping framework in Spring Boot applications.

Last Release on Dec 28, 2025

Scope: Compile Format: Kotlin

```
// Source: https://mvnrepository.com/artifact/org.mybatis.spring.boot/mybatis-spring-boot-starter
implementation("org.mybatis.spring.boot:mybatis-spring-boot-starter:4.0.1")
```



➤ Mybatis 의존성 추가

- build.gradle 추가한다.

MyBatis의 경우 버전을 삭제하면 안된다.

MyBatis는 spring boot에서 관리하지 않고 Org.mybatis에서 독자적으로 관리하므로 버전명을 명시하지 않으면 error가 발생할 수 있음을 주의하자!!

```
*build.gradle x
15
16
17 repositories {
18     mavenCentral()
19 }
20
21 dependencies {
22     implementation 'org.springframework.boot:spring-boot-starter-jdbc'
23     implementation 'org.springframework.boot:spring-boot-starter-security'
24     //implementation 'org.springframework.boot:spring-boot-starter-jdbc'
25
26     // Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-jdbc
27     implementation("org.springframework.boot:spring-boot-starter-jdbc")
28
29     // Source: https://mvnrepository.com/artifact/com.mysql/mysql-connector-j
30     implementation("com.mysql:mysql-connector-j")
31
32     // Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-security
33     implementation("org.springframework.boot:spring-boot-starter-security")
34
35     // Source: https://mvnrepository.com/artifact/org.mybatis.spring.boot/mybatis-spring-boot-starter
36     implementation("org.mybatis.spring.boot:mybatis-spring-boot-starter:4.0.1")
37
38     developmentOnly 'org.springframework.boot:spring-boot-devtools'
39     testImplementation 'org.springframework.boot:spring-boot-starter-thymeleaf-test'
40     testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
41     testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
42 }
```

> mybatis-3.5.19.jar - C:\Users\Admin\gradle\caches\modules-2\files-2.1\org.mybatis\mybatis\3.5.19\mybatis-3.5.19.jar

> mybatis-spring-4.0.0.jar - C:\Users\Admin\gradle\caches\modules-2\files-2.1\org.mybatis.spring.boot\mybatis-spring\4.0.0\mybatis-spring-4.0.0.jar

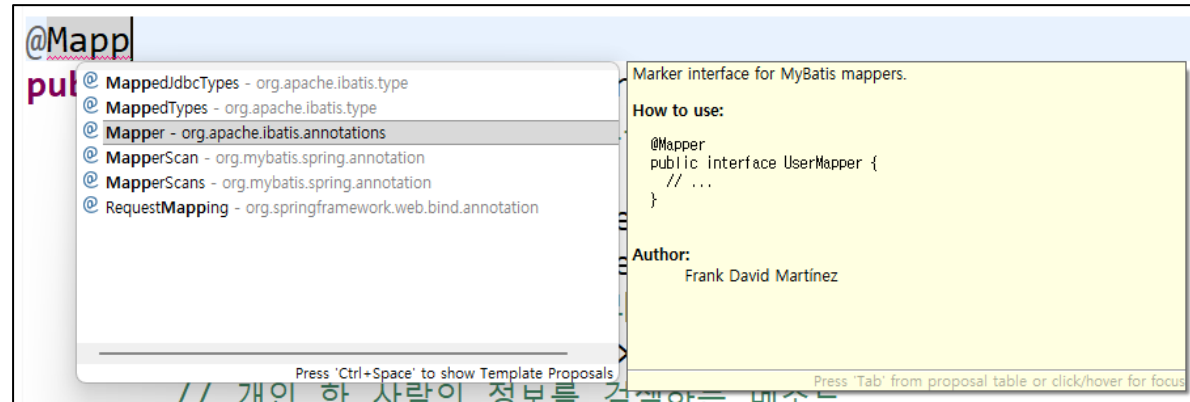
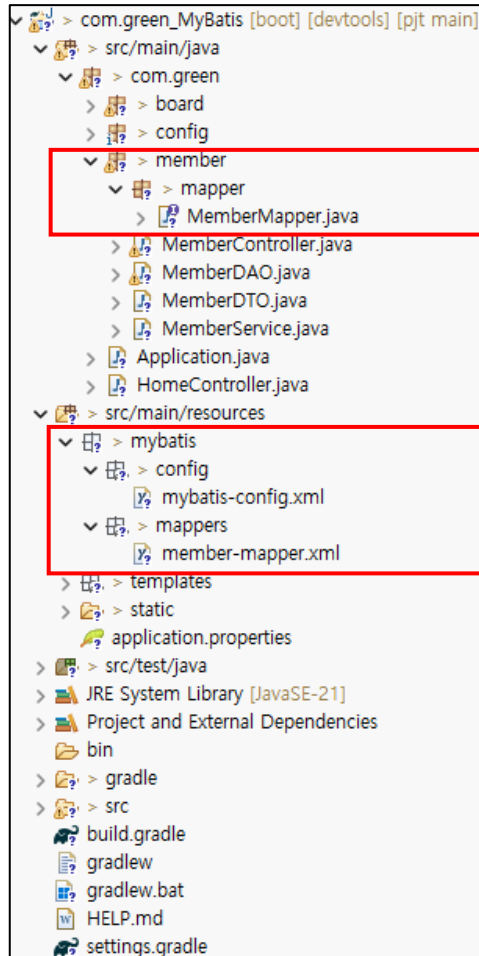
> mybatis-spring-boot-autoconfigure-4.0.1.jar - C:\Users\Admin\gradle\caches\modules-2\files-2.1\org.mybatis.spring.boot\mybatis-spring-boot-autoconfigure\4.0.1\mybatis-spring-boot-autoconfigure-4.0.1.jar

> mybatis-spring-boot-starter-4.0.1.jar - C:\Users\Admin\gradle\caches\modules-2\files-2.1\org.mybatis.spring.boot\mybatis-spring-boot-starter\4.0.1\mybatis-spring-boot-starter-4.0.1.jar



➤ 매퍼 인터페이스 생성

- src/main/java → member 패키지 → mapper 패키지 → MemberMapper 인터페이스를 생성한다.





➤ 매퍼 인터페이스 생성

- src/main/java → member 패키지 → mapper 패키지 → MemberMapper 인터페이스를 생성한다.
- MemberMapper 인터페이스에 MemberDAO에서 작성한 메소드들을 추상 메소드로 작성한다.

```
package com.green.member.mapper;

import java.util.List;

import org.apache.ibatis.annotations.Mapper;

@Mapper
public interface MemberMapper {
    // MemberDAO의 메소드들을 모두 추상메소드로 작성한다.
    // 이렇게 설정된 객체는 IoC 스프링 컨테이너에 탑재가 된다.

    // 회원 개인을 추가하는 메소드
    public int insertMember(MemberDTO mdto);
    public boolean isMember(String id);
    // 회원 전체 목록 검색 쿼리
    public List<MemberDTO> allSelectMember();
    // 개인 한 사람의 정보를 검색하는 메소드
    public MemberDTO oneSelectMember(String id);
    // 개인 한사람의 정보를 수정하는 쿼리
    public int updateMember(MemberDTO mdto);
    // 개인 한사람의 비밀번호 리턴하는 쿼리
    public String getPass(String id);
    // 한사람 개인의 정보를 삭제하는 메소드 작성
    public int deleteMember(String id);
}
```



➤ 매퍼 인터페이스 생성

- src/main/java → member 패키지 → mapper 패키지 → MemberMapper 인터페이스를 생성한다.
- MemberService 클래스에서 MemberMapper 인터페이스를 의존객체 주입한다.

```
@Service
public class MemberService {

    // id중복체크, 성공, 실패 상수변수 정의
    // 회원가입의 중복을 확인하는 상수
    public final static int user_id_alreday_exit = 0;
    // 회원가입의 성공여부를 확인하는 상수
    public final static int user_id_success = 1;
    // 회원가입의 실패를 확인하는 상수
    public final static int user_id_fail = -1;

    // 1. MemberDAO 대신 MemberMapper를 주입받습니다.
    @Autowired
    private MemberMapper memberMapper; // 변수명을 관례에 맞게 memberMapper로 수정

    //암호화 하는 PasswordEncoder도 의존성 객체로 주입한다.
    @Autowired
    PasswordEncoder passwordencoder;
```



➤ XML 매퍼 생성

- src/main/resources → mybatis 폴더 → config 폴더 → **mybatis-config.xml** 파일을 생성한다.
- **Mybatis 환경을 설정하는 파일이다.**
- **mybatis-config.xml** 파일에 아래 코드를 작성한다.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
"https://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

</configuration>
```




➤ XML 매퍼 생성

- src/main/resources → mybatis 폴더 → mappers → **member-mapper.xml** 파일을 생성한다.
- **SQL 문을 작성하는 파일이다.**
- **member-mapper.xml** 파일에 아래 코드를 작성한다.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper>

</mapper>
```



➤ Mybatis 적용

MemberMapper 인터페이스

```
// 회원 개인을 추가하는 메소드
public int insertMember(MemberDTO mdto);

//id 없음 => false
public boolean isMember(String id);

// 회원 전체 목록 검색 쿼리
public List<MemberDTO> allSelectMember();

// 개인 한 사람의 정보를 검색하는 메소드
public MemberDTO oneSelectMember(String id);

// 개인 한 사람의 정보를 수정하는 쿼리
public int updateMember(MemberDTO mdto);

// 개인 한 사람의 패스워드 리턴하는 쿼리
public String getPass(String id);

// 한 사람 개인의 정보를 삭제하는 메소드 작성
public int deleteMember(String id);
```

member-mapper.xml

```
<insert id="insertMember">

<select id="isMember">

<select id="allSelectMember">

<select id="oneSelectMember">

<update id="updateMember">

<select id="getPass">

<delete id="deleteMember">
```



➤ XML 매퍼 생성

- src/main/resources → mybatis 폴더 → mappers → **member-mapper.xml** 파일을 생성한다.

```
<!-- insert문이나, update문은 resultType를 적지 않는다. -->
<insert id="insertMember"
  parameterType="com.green.member.MemberDTO">
  INSERT INTO user_member(id,pw,mail,phone) VALUES(#{id},#{pw},#{mail},#{phone})
</insert>

<select id="isMember" parameterType="string" resultType="boolean">
  SELECT COUNT(*) FROM user_member WHERE id = #{id}
</select>

<select id="allSelectMember"
  resultType="com.green.member.MemberDTO"
>
  SELECT * FROM user_member
</select>

<select id="oneSelectMember"
  resultType="com.green.member.MemberDTO"
  parameterType="String"
>
  SELECT * FROM user_member WHERE id=#{id}
</select>
```



➤ XML 매퍼 생성

- src/main/resources → mybatis 폴더 → mappers → **member-mapper.xml** 파일을 생성한다.

```
<select id="getPass" resultType="String"
parameterType="String"
>
SELECT pw FROM user_member WHERE id=#{id}
</select>

<delete id="deleteMember"
parameterType="String"
>
DELETE FROM user_member WHERE id=#{id}
</delete>

<update id="updateMember"
parameterType="com.green.member.MemberDTO"
>
    UPDATE user_member SET mail=#{mail},phone=#{phone} WHERE id=#{id}
</update>
```



➤ XML 매퍼 생성

- src/main/resources → mybatis 폴더 → config → **mybatis-config.xml** 파일을 생성한다.
- mybatis-config.xml은 MyBatis의 **전역 설정(Global Configuration)**을 담당하는 파일이다.

1. 앨리어스(Alias) 설정

- 패키지 경로가 긴 클래스명(예: com.green.member.MemberDTO)을 XML에서 짧은 별명(memberDTO)으로 사용할 수 있게 등록한다.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "https://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
<typeAliases>
    <typeAlias alias="MemberDTO" type="com.green.member.MemberDTO"/>
</typeAliases>
</configuration>
```



➤ XML 매퍼 생성

- src/main/resources → mybatis 폴더 → mappers → member-mapper.xml 파일을 mybatis-config.xml에서 정의한 별칭(Alias)인 MemberDTO로 모두 변경한다.

```
<!-- insert문이나, update문은 resultType를 적지 않는다. -->
<insert id="insertMember"
  parameterType="MemberDTO">
  INSERT INTO user_member(id,pw,mail,phone) VALUES("#{id}",#{pw},#{mail},#{phone})
</insert>

<select id="isMember" parameterType="string" resultType="boolean">
  SELECT COUNT(*) FROM user_member WHERE id = #{id}
</select>

<select id="allSelectMember"
  resultType="MemberDTO"
>
  SELECT * FROM user_member
</select>

<select id="oneSelectMember"
  resultType="MemberDTO"
  parameterType="String"
>
  SELECT * FROM user_member WHERE id=#{id}
</select>
```



➤ XML 매퍼 생성

- src/main/resources → mybatis 폴더 → mappers → **member-mapper.xml** 파일을 **mybatis-config.xml**에 정의한 별칭(Alias)인 **MemberDTO**로 모두 변경한다.

```
<select id="getPass" resultType="String"
parameterType="String"
>
SELECT pw FROM user_member WHERE id=#{id}
</select>

<delete id="deleteMember"
parameterType="String"
>
DELETE FROM user_member WHERE id=#{id}
</delete>

<update id="updateMember"
parameterType="MemberDTO"
>
    UPDATE user_member SET mail=#{mail},phone=#{phone} WHERE id=#{id}
</update>
```



➤ Application.properties 파일에 Mybatis 부분을 추가한다.

```
spring.application.name=com.green_MyBatis
# Tomcat 서버를 내장되어 있다.
# 원래 포트번호는 8080인데 => 8090으로 변경함
# Tomcat
server.port=8090

# Dev Tools
spring.devtools.restart.enabled=true

# Thymeleaf #주석
spring.thymeleaf.cache=false
spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html
spring.thymeleaf.check-template-location=true

# MySQL
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/springBootDB
spring.datasource.username=root
spring.datasource.password=12345678
```

```
# Mybatis
mybatis.config-location=classpath:mybatis/config/mybatis-config.xml
mybatis.mapper-locations=classpath:mybatis/mappers/*.xml
```

```
# [추가] MyBatis SQL 로그 출력 설정
# 패키지 경로(com.green.member.mapper)의 로그 레벨을 DEBUG로 설정하면
# 실행되는 SQL문과 전달되는 파라미터(? 값)를 콘솔에서 볼 수 있습니다.
logging.level.com.green.member.mapper=DEBUG
```

```
# [선택] 만약 스프링 부트의 전반적인 DB 연결 상태나 설정 로그를 보고 싶다면 아래도 추가하세요.
# logging.level.org.springframework.jdbc=DEBUG
```




➤ XML 매퍼 생성

- src/main/resources → mybatis 폴더 → config → **mybatis-config.xml** 파일을 생성한다.
- mybatis-config.xml은 MyBatis의 **전역 설정(Global Configuration)**을 담당하는 파일이다.

2. XML 매퍼 파일 등록

- MyBatis에게 "실제 SQL 문이 작성된 설계도(XML)가 어디에 있는지" 위치를 알려주는 것을 의미한다.
- 기존에 작성한 MemberMapper.xml과 같은 파일들이 제 기능을 하려면 MyBatis가 이 파일들을 읽어 들여야 하는데, 그 연결 통로를 설정하는 작업이다.
- Application.properties 파일의 Mybatis 부분의 mybatis.mapper-locations=classpath:mybatis/mappers/*.xml => 이 부분을 아래처럼 mybatis-config.xml에 작성할 수 있다.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "https://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

  <typeAliases>
    <typeAlias alias="MemberDTO" type="com.green.member.MemberDTO"/>
  </typeAliases>
  <mappers>
    <mapper resource="mybatis/mappers/member-mapper.xml"/>
  </mappers>
</configuration>
```

spring MVC

MyBatis 이용

게시판 수정하기





➤ BoardService 클래스를 인터페이스로 수정한다.

- src/main/java → board 폴더 → BoardService 인터페이스 파일을 생성한 후 아래 처럼 작성한다.
- 단, 모두 추상메소드로만 작성한다.

```
package com.green.board;

import java.util.List;

public interface BoardService {
    // 하나의 게시글 추가
    public void addBoard(BoardDTO bdto);

    // 전체 게시글 출력
    public List<BoardDTO> allBoard();

    // 하나의 게시글 상세 조회
    public BoardDTO oneBoard(int num);

    // 게시글 클릭할때 마다 조회수 1씩 증가하는 메소드 작성
    public int updateReadCount(int num);

    // 게시글 수정
    public boolean modifyBoard(BoardDTO bdto);

    // 게시글 삭제
    public boolean removeBoard(int num, String writerPw);

    // 게시글 검색
    public List<BoardDTO> searchBoard(String searchType, String searchKeyword);
}
```



➤ BoardService 클래스를 인터페이스로 수정한다.

- src/main/java → board 폴더 → BoardService 인터페이스 파일을 상속(**implements**)받을 BoardServiceImpl 파일을 생성한 후 아래 처럼 작성한다.

```
package com.green.board;

import java.util.List;

@Service
public class BoardServiceImpl implements BoardService {

    @Autowired
    private BoardMapper boardMapper; // MyBatis 주입

    @Override
    public void addBoard(BoardDTO bdto) {
        boardMapper.insertBoard(bdto);
    }

    @Override
    public List<BoardDTO> allBoard() {
        return boardMapper.getAllBoard();
    }

    @Override
    public BoardDTO oneBoard(int num) {
        System.out.println("BoardServiceImpl oneBoard() 메소드 확인");

        // 1. 먼저 조회수를 1 증가시킵니다.
        boardMapper.updateReadCount(num);
        // 게시글 상세 정보로 넘긴다.
        return boardMapper.getOneBoard(num);
    }
}
```

게시글 하나를 상세 정보로 넘기기 전에 조회수가 1이 증가되어야 하므로 updateReadCount(num)메소드를 먼저 실행시킨다.



➤ BoardService 클래스를 인터페이스로 수정한다.

- src/main/java → board 폴더 → BoardService 인터페이스 파일을 상속(**implements**)받을 BoardServiceImpl 파일을 생성한 후 아래 처럼 작성한다.

```
@Override
public boolean modifyBoard(BoardDTO bdto) {
    System.out.println("BoardServiceImpl modifyBoard() 메소드 확인");
    int result = boardMapper.updateBoard(bdto);

    if(result > 0) {
        System.out.println("게시글 수정 성공");
        return true;
    } else {
        System.out.println("게시글 수정 실패");
        return false;
    }
}

@Override
public boolean removeBoard(int num, String writerPw) {
    System.out.println("BoardServiceImpl removeBoard() 메소드 확인");
    int result = boardMapper.deleteBoard(num, writerPw);

    if(result > 0) {
        System.out.println("게시글 삭제 성공");
        return true;
    } else {
        System.out.println("게시글 삭제 실패");
        return false;
    }
}
```



➤ BoardService 클래스를 인터페이스로 수정한다.

- src/main/java → board 폴더 → BoardService 인터페이스 파일을 상속(**implements**)받을 BoardServiceImpl 파일을 생성한 후 아래 처럼 작성한다.

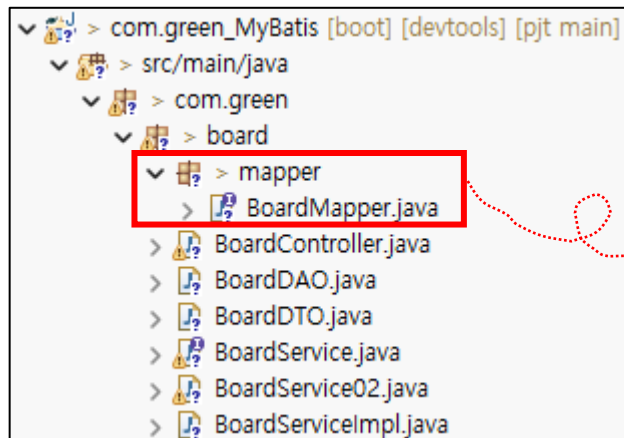
```
@Override
public List<BoardDTO> searchBoard(String searchType, String searchKeyword) {
    System.out.println("BoardServiceImpl searchBoard() 호출: " + searchType + " / " + searchKeyword);
    return boardMapper.getSearchBoard(searchType, searchKeyword);
}
```

```
@Override
public int updateReadCount(int num) {
    System.out.println("BoardServiceImpl updateReadCount() 메소드 확인");
    return boardMapper.updateReadCount(num);
}
```



➤ 매퍼 인터페이스 생성

- src/main/java → board 패키지 → mapper 패키지 → BoardMapper 인터페이스를 생성한다.
- 기존 BoardDAO 를 이용해 BoardMapper 인터페이스를 작성한다.



```
package com.green.board.mapper;

import java.util.List;

@Mapper
public interface BoardMapper {

    // 하나의 게시글을 추가하는 메소드 -----
    public void insertBoard(BoardDTO bdto);

    // 전체 게시글을 검색하는 메소드 -----
    public List<BoardDTO> getAllBoard();

    // 하나의 게시글을 리턴받는 메소드 작성
    public BoardDTO getOneBoard(int num);

    // 하나의 게시글 수정하는 메소드 -----
    public int updateBoard(BoardDTO bdto);

    // 하나의 게시글을 삭제하는 메소드 -----
    //public int deleteBoard(int num, String writerPw);
    // @Param을 통해 XML에서 사용할 이름을 명시적으로 지정합니다.
    public int deleteBoard(@Param("num") int num, @Param("writerPw") String writerPw);

    // 내용 또는 제목으로 게시글 검색하는 메소드
    // 검색메소드 반드시, searchType, searchKeyword 매개변수 필요
    //public List<BoardDTO> getSearchBoard(String searchType, String searchKeyword);
    // 검색 조건(타입)과 키워드를 전달받습니다.
    public List<BoardDTO> getSearchBoard(@Param("searchType") String searchType,
                                         @Param("searchKeyword") String searchKeyword);
}
```



➤ Mybatis-config.xml 환경 설정

- src/main/resources → mybatis 패키지 → config 패키지 → mybatis-config.xml 파일을 생성 후 아래처럼 전역 환경설정을 작성한다.
- BoardDTO 의 별칭(typeAlias)을 설정한다.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "https://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

  <!-- <settings>
    <setting name="cacheEnabled" value="true" />
  </settings> -->

  <typeAliases>
    <typeAlias alias="MemberDTO" type="com.green.member.MemberDTO"/>
    <typeAlias alias="BoardDTO" type="com.green.board.BoardDTO"/>
  </typeAliases>

  <!-- <mappers>
    <mapper resource="mybatis/mappers/member-mapper.xml"/>
  </mappers> -->

</configuration>
```




➤ Board-mapper.xml 파일 생성 및 작성

- src/main/resources → mybatis 패키지 → mappers 패키지 → board-mapper.xml 파일을 생성한다.
- 기존에 작성한 BoardDAO를 이용해서 board-mapper.xml을 작성한다.

```
▼ > src/main/resources
  ▼ > mybatis
    ▼ > config
      mybatis-config.xml
    ▼ > mappers
      board-mapper.xml
      member-mapper.xml
  > templates
  ▼ > static
    > css
    > img
    > js
    > sql
  application.properties
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTO Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.green.board.mapper.BoardMapper">

  <insert id="insertBoard" parameterType="BoardDTO">
    INSERT INTO board(writer,subject,writerPw,content,id)
    VALUES(#{writer},#{subject},#{writerPw},#{content},#{id})
  </insert>

  <select id="getAllBoard" resultType="BoardDTO">
    SELECT * FROM board ORDER BY num DESC
  </select>

  <update id="updateReadCount" parameterType="int">
    UPDATE board SET readcount = readcount + 1 WHERE num = #{num}
  </update>

  <select id="getOneBoard" parameterType="int" resultType="BoardDTO">
    SELECT * FROM board WHERE num = #{num}
  </select>

</mapper>
```



➤ Board-mapper.xml 파일 생성 및 작성

- src/main/resources → mybatis 패키지 → mappers 패키지 → board-mapper.xml 파일을 생성한다.

```
<update id="updateBoard" parameterType="BoardDTO">
    UPDATE board SET subject=#{subject}, content=#{content}
    WHERE num=#{num} AND writerPw=#{writerPw}
</update>

<delete id="deleteBoard">
    DELETE FROM board
    WHERE num = #{num} AND writerPw = #{writerPw}
</delete>

<select id="getSearchBoard" resultType="BoardDTO">
    SELECT * FROM board
    <where>
        <choose>
            <when test="searchType == 'subject'">
                subject LIKE CONCAT('%', #{searchKeyword}, '%')
            </when>
            <otherwise>
                content LIKE CONCAT('%', #{searchKeyword}, '%')
            </otherwise>
        </choose>
    </where>
    ORDER BY num DESC
</select>
```

```
</mapper>
```



- DB – board 테이블 수정하고 새로 생성
 - 기존 board 테이블을 아래와 같이 수정하여 새로 생성한다.

```
create table board(  
  num int auto_increment primary key,  
  writer varchar(20),  
  subject varchar(30),  
  writerPw varchar(20),  
  reg_date datetime default now(),  
  readcount int default 0,  
  content varchar(1000),  
  id varchar(20)  
);
```



➤ boardList.html 파일 수정

- 기존 boardList.html 파일의 게시글의 [글쓰기]버튼을 클릭하면 “로그인 후 이용하세요”라는 메시지가 출력되도록 작성하고, 로그인 상태에서 만 게시글의 글쓰기가 되도록 코드를 수정한다.

```
<table width="700" border="1">
  <tr height="40">
    <td colspan="5" align="right">
      <!-- <a th:href="@{/board/write}" class="write-btn">글쓰기</a> -->
      <a href="#" id="writeBtn" class="write-btn"
        th:data-login="${session.LoginedMember != null}">
        글쓰기
      </a>
    </td>
  </tr>
</table>
```



➤ boardList.html 파일 수정

- 기존 boardList.html 파일에 연결된 member.js 에 아래의 코드를 작성한다.

단, javaScript의 특성상 HTML이 생성된 후 javaScript가 실행 되도록 아래 코드를 수정 하시오.

```
<!-- "defer는 'HTML 다 만든 다음에 JS 실행해!'라는 뜻" -->
<script th:src="@{/js/member.js}" defer></script>
```

- member.js 에 아래의 코드를 추가 작성한다.

```
// 현재 로그인상태이면 게시글의 글쓰기 작성
let write = document.getElementById("writeBtn");

write.addEventListener("click", function () {

    const isLogin = this.dataset.login; // "true" or "false"

    if (isLogin === "true") {
        location.href = "/board/write";
    } else {
        alert("로그인 후 이용 가능합니다.");
        location.href = "/member/login";
    }

});
```



➤ BoardController 클래스를 수정한다.

```
@Controller
public class BoardController {

    //BoardServiceImpl 말고 BoardService 인터페이스를 주입받는것이
    // 실무 관례이다.
    @Autowired
    BoardService boardService;
```

BoardService 인터페이스를 의존객체로 삽입함을 주의하자!!

- 게시판에 글쓰기 할 때 세션에서 로그인 된 회원의 정보를 가져와야 한다.

```
// 2. 게시글 저장 처리 (신규 추가 중요!)
@PostMapping("/board/writePro")
public String boardWritePro(BoardDTO bdto, HttpSession session) {
    System.out.println("BoardController boardWritePro() 호출");

    //1. 세션에서 로그인된 회원정보(MemberDTO)를 가져온다.
    //다운캐스팅한다.
    MemberDTO loggedMember = (MemberDTO)session.getAttribute("loggedMember");

    // 2. 로그인 정보가 있는지 반드시 체크합니다. (데이터 누락 방지)
    if (loggedMember != null) {
        // [중요] 세션에서 꺼낸 id를 DTO에 수동으로 넣어줘야 MyBatis가 인식합니다.
        bdto.setId(loggedMember.getId());
        System.out.println("DB에 저장될 ID 확인: " + loggedMember.getId());
    } else {
        System.out.println("로그인 세션이 없습니다!");
        return "redirect:/member/login";
    }

    // id가 포함된 bdto를 서비스로 넘겨 addBoard 호출하여 DB 저장
    boardService.addBoard(bdto);

    // 저장 후에는 '목록' 페이지로 이동 (Redirect)
    return "redirect:/board/list";
}
```